Unit - I

Fundamentals of Algorithm Analysis

What is an algorithm?

Muhammad ibn Mūsā al-Khwārizmī – 9th century Persian Muslim Mathematician

"algorism" referred to rules of performing arithmetic using Arabic numerals

Evolved into "algorithm" and came to mean a definite procedure for solving a problem or performing a task

"the idea behind any reasonable computer program"

Something like a recipe, process, method, technique, procedure, routine etc

What is an algorithm?

There is no formal definition of an algorithm.

The following features make a sensible informal definition:

1. **Finiteness**: Must terminate after a (very) finite number of steps.
2. **Definiteness**: Each step must be precisely defined (rigourous, unambiguous). Programming languages are designed for specifying algorithms – every statement has a definite meaning.
3. **Input**: One or more inputs.
   Q: Are there any useful algorithms with no inputs?
4. **Output**: One or more outputs.
   Q: Are there any useful algorithms with no outputs?
5. **Effectiveness**: Operations must be sufficiently basic that they can be done exactly and in a finite length of time, i.e. computable or do-able.

What is an algorithm?

There is no formal definition of an algorithm.

The following features make a sensible informal definition:

1. **Finiteness**: Must terminate after a (very) finite number of steps.
2. **Definiteness**: Each step must be precisely defined (rigourous, unambiguous). Programming languages are designed for specifying algorithms – every statement has a definite meaning.
3. **Input**: One or more inputs.
   Q: Are there any useful algorithms with no inputs?
4. **Output**: One or more outputs.
   Q: Are there any useful algorithms with no outputs?
5. **Effectiveness**: Operations must be sufficiently basic that they can be done exactly and in a finite length of time, i.e. computable or do-able.

What is an algorithm?

There is no formal definition of an algorithm.

The following features make a sensible informal definition:

1. **Finiteness**: Must terminate after a (very) finite number of steps.
2. **Definiteness**: Each step must be precisely defined (rigourous, unambiguous). Programming languages are designed for specifying algorithms – every statement has a definite meaning.
3. **Input**: One or more inputs.
   Q: Are there any useful algorithms with no inputs?
4. **Output**: One or more outputs.
   Q: Are there any useful algorithms with no outputs?
5. **Effectiveness**: Operations must be sufficiently basic that they can be done exactly and in a finite length of time, i.e. computable or do-able.

What is an algorithm?

There is no formal definition of an algorithm.

The following features make a sensible informal definition:

1. **Finiteness**: Must terminate after a (very) finite number of steps.
2. **Definiteness**: Each step must be precisely defined (rigourous, unambiguous). Programming languages are designed for specifying algorithms – every statement has a definite meaning.
3. **Input**: One or more inputs.
   Q: Are there any useful algorithms with no inputs?
4. **Output**: One or more outputs.
   Q: Are there any useful algorithms with no outputs?
5. **Effectiveness**: Operations must be sufficiently basic that they can be done exactly and in a finite length of time, i.e. computable or do-able.

What is an algorithm?

There is no formal definition of an algorithm.

The following features make a sensible informal definition:

1. **Finiteness**: Must terminate after a (very) finite number of steps.
2. **Definiteness**: Each step must be precisely defined (rigourous, unambiguous). Programming languages are designed for specifying algorithms – every statement has a definite meaning.
3. **Input**: One or more inputs.
   Q: Are there any useful algorithms with no inputs?
4. **Output**: One or more outputs.
   Q: Are there any useful algorithms with no outputs?
5. **Effectiveness**: Operations must be sufficiently basic that they can be done *exactly* and in a *finite length of time*, i.e. computable or do-able.

What is an algorithm?

According to our criteria, are the following valid algorithms?

Find the greatest common divisor of positive integers m and n

1. If m<n, exchange m and n
2. Divide m by n and let r be the remainder
3. If r = 0, terminate and return n as the answer
4. Set m = n and n = r, return to 1

What is an algorithm?

According to our criteria, are the following valid algorithms?

Find the greatest common divisor of positive integers m and n

1. If m<n, exchange m and n
2. Divide m by n and let r be the remainder
3. If r = 0, terminate and return n as the answer
4. Set m = n and n = r, return to 1

Valid

This is our first algorithm

One of the oldest known algorithms (300 BC) described by Euclid (in a slightly different form)

What is an algorithm?

In practice we want algorithms that are:

- Correct
- Efficient
- Easy to implement

May not be simultaneously achievable

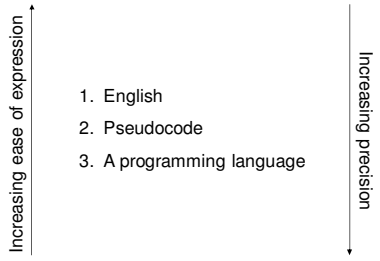
Specifications and correctness

- An algorithm is step-wise representation of a solution to a problem
- We need a way to specify a problem so we can design algorithms to solve it
- A computer program is a translation of an algorithm into a particular programming language (also an *implementation*)
- Checking that an implementation correctly executes an algorithm is the realms of software engineering and formal methods
- We are interested in the analysis of algorithms
- We don't care what language it is subsequently implemented in
- But we do need a language to express algorithms

Q: Why don't we always use English? Why don't we program computers in English?

Expressing Algorithms

Three possibilities:



Expressing Algorithms

- We use a mixture of English & standard keywords of programming language - *pseudocode*
- Nice features of pseudocode:
  - Pseudocode represents a good balance between precision and readability
  - It is straightforward to translate to any programming language
  - It's intuitive - no need to learn it

Pseudocode Example

- The algorithm for finding the greatest common divisor given above can be expressed in pseudocode as follows:

Function $GCD(m, n : \mathbb{N}) : \mathbb{N}$
1: **if** $m < n$ **then**
2: (m,n):=(n,m)
3: **end if**
4: **while** $n \neq 0$ **do**
5: $r := m$ **mod** n
6: $(m, n) := (n, r)$
7: **end while**
8: **return** m

- **Exercise**: try it out – for different input values, trace the operation of the algorithm keeping track of current values of the variables

Specifications and correctness

As algorists, we need tools to:

- Specify a problem so we can design algorithms to solve it
- Distinguish correct algorithms from incorrect ones
- Compare algorithms and choose the "best" one

Specifications and correctness

As algorists, we need tools to:

- Specify a problem so we can design algorithms to solve it
  - and strategies to design algorithms which satisfy specifications
- Distinguish correct algorithms from incorrect ones
- Compare algorithms and choose the "best" one

Specifications and correctness

As algorists, we need tools to:

- Specify a problem so we can design algorithms to solve it
  - Commonly solve reduced version of problem if initially too difficult
- Distinguish correct algorithms from incorrect ones
  - Check that the algorithm satisfies the problem specification
- Compare algorithms and choose the "best" one

Specifications and correctness

As algorists, we need tools to:

- Specify a problem so we can design algorithms to solve it
  - Commonly solve reduced version of problem if initially too difficult
- Distinguish correct algorithms from incorrect ones
  - Check that the algorithm satisfies the problem specification
- Compare algorithms and choose the "best" one
  - There may be many alternative algorithms that satisfy a specification
  - We want the one that is "best" for our purposes
  - Often, best = most efficient. Is this always the case?
  - The subject of much of this course!

Analysis of Algorithms

*The theoretical study of computer-program **performance** and resource usage.*

Is performance the most important measure of an algorithm?

What's more important than performance?

Analysis of Algorithms

*The theoretical study of computer-program **performance** and **resource usage**.*

Is performance the most important measure of an algorithm?

What's more important than performance?

- correctness
- security
- user-friendliness
- maintainability
- robustness/reliability
- functionality/features
- simplicity
- programmer time
- modularity
- extensibility

Performance is the ***currency*** of computation

It can be used to measure other things: functionality, user-friendliness etc.

Algorithm Performance

Q: How might we establish whether algorithm A is faster than algorithm B?

Algorithm Performance

Q: How might we establish whether algorithm A is faster than algorithm B?

A1: We could implement both of them, run them on the same input and time how long each of them takes

Algorithm Performance

Q: How might we establish whether algorithm A is faster than algorithm B?

A1: We could implement both of them, run them on the same input and time how long each of them takes

Unfair test: what if one of the algorithms just happens to be faster on this particular input?

Algorithm Performance

Q: How might we establish whether algorithm A is faster than algorithm B?

A2: We could implement both of them, run them on lots of different inputs and time how long each of them takes on each input

Algorithm Performance

Q: How might we establish whether algorithm A is faster than algorithm B?

A2: We could implement both of them, run them on lots of different inputs and time how long each of them takes on each input

Assuming we can try every input of a particular size, this would give us best, worst and average running times for this *particular implementation* on this *particular computer* for this *particular input size*

Still an unfair test: what if one algorithm just happens to be faster on other ***size*** of input?

What if we want a more general answer? Not tied to one computer or implementation.

Algorithm Performance

Let's generalise things slightly…

The function: $T : I \rightarrow R_+$

is a *mapping* from the set of all inputs to the time taken on that input

For any problem instance: $i \in I$

$T(i)$ is the running time on i

- Computing the running time for every possible problem instance is overwhelming
- Instead, group together "similar" inputs
- It gives us running time as a function of a class of instances
- How shall we group inputs?

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Worst-case: $T(n) = \max\{T(i) : i \in I_n\}$

- $T(n)$ = maximum time of algorithm on any input of size n.

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Worst-case: $T(n) = \max\{T(i) : i \in I_n\}$

- $T(n)$ = maximum time of algorithm on any input of size n.

Best-case: $T(n) = \min\{T(i) : i \in I_n\}$

- $T(n)$ = minimum time of algorithm on any input of size n.

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Worst-case: $T(n) = \max\{T(i) : i \in I_n\}$

- $T(n)$ = maximum time of algorithm on any input of size n.

Best-case: $T(n) = \min\{T(i) : i \in I_n\}$

- $T(n)$ = minimum time of algorithm on any input of size n.

Average-case: $T(n) = \frac{1}{|I_n|} \sum_{i \in I_n} T(i)$

- $T(n)$ = expected time of algorithm over all inputs of size n.

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Worst-case: $T(n) = \max\{T(i) : i \in I_n\}$

- $T(n)$ = maximum time of algorithm on any input of size n.

Best-case: $T(n) = \min\{T(i) : i \in I_n\}$

- $T(n)$ = minimum time of algorithm on any input of size n.

Average-case: $T(n) = \frac{1}{|I_n|}\sum_{i \in I_n} T(i)$

- $T(n)$ = expected time of algorithm over all inputs of size n.

Q: What assumption is being made here?

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:

Worst-case: $T(n) = \max\{T(i) : i \in I_n\}$

- $T(n)$ = maximum time of algorithm on any input of size n.

Best-case: $T(n) = \min\{T(i) : i \in I_n\}$

- $T(n)$ = minimum time of algorithm on any input of size n.

Average-case: $T(n) = \frac{1}{|I_n|}\sum_{i \in I_n} T(i)$

- $T(n)$ = expected time of algorithm over all inputs of size n.

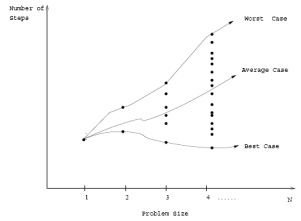
Q: What assumption is being made here?

All inputs *equally likely* – if not, we need to know the probability distribution

Types of performance analysis

We denote the set of all instances of size $n \in N$ as I_n

We can define three measures of performance:



Q: Which is most useful?

Q: How can we modify almost any algorithm to have a good best-case running time?

Types of performance analysis

Q: Which is most useful?

A: Generally concentrate on *worst-case execution time* – *strongest performance guarantee*

Types of performance analysis

Q: Which is most useful?

A: Generally concentrate on worst-case execution time – strongest performance guarantee

Q: How can we modify almost any algorithm to have a good best-case running time?

A: Find a solution for one particular input and store it. When that input is encountered, return our precomputed answer immediately.

Other more subtle ways of improving best-case performance.

Best-case is generally bogus!

Measuring resource usage

Example

Summing the first n positive integers:

$$\text{Precondition}: n \in N_+; \quad \text{Postcondition}: r = \sum_{i=1}^{n} i$$

Two solutions:

```
r := 0
for i := 1 to n do
   r := r + i
end for
```

$$r = \frac{n(n+1)}{2}$$

Measuring resource usage

Example

Summing the first n positive integers:

$$\text{Precondition}: n \in N_+; \quad \text{Postcondition}: r = \sum_{i=0}^{n} i$$

Two solutions:

```
r := 0
for i := 1 to n do
   r := r + i
end for
```

$$r = \frac{n(n+1)}{2}$$

Both algorithms are correct

Q: Which is better?

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 1 Cost: No. Times:

```
r := 0
for i := 1 to n do
   r := r + i
end for
```


Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 1	Cost:	No. Times:
$r := 0$	s	1
for $i := 1$ **to** n **do**		
$\quad r := r + i$		
end for		

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 1	Cost:	No. Times:
$r := 0$	s	1
for $i := 1$ **to** n **do**	$t+i$	$n+1$
$\quad r := r + i$		
end for		

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 1	Cost:	No. Times:
$r := 0$	s	1
for $i := 1$ **to** n **do**	$t+i$	$n+1$
$\quad r := r + i$	$a+s$	n
end for		

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 1	Cost:	No. Times:
$r := 0$	s	1
for $i := 1$ **to** n **do**	$t+i$	$n+1$
$\quad r := r + i$	$a+s$	n
end for		

$$T1 = \lambda n.n(t + i + a + s) + t + i + s$$

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 2	Cost:	No. Times:
$r = \frac{n(n+1)}{2}$		

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

Version 2	Cost:	No. Times:
$r = \frac{n(n+1)}{2}$	$i+m+d+s$	1

Measuring resource usage

Define some constants:

- i is the time to increment by 1
- a is the time to perform an addition
- t is the time to perform the loop test
- m is the time to multiply two numbers
- d is the time to divide by 2
- s is the time to perform an assignment

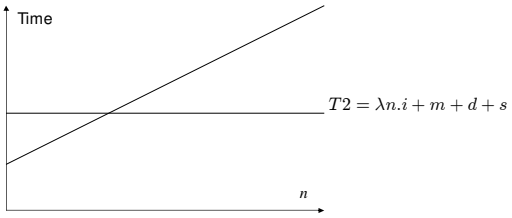
Version 2	Cost:	No. Times:
$r = \frac{n(n+1)}{2}$	$i+m+d+s$	1

$$T2 = \lambda n.i + m + d + s$$

Measuring resource usage

Which is better?

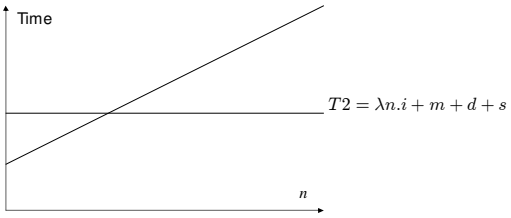
$$T1 = \lambda n.n(t + i + a + s) + t + i + s$$



Measuring resource usage

Which is better?

$$T1 = \lambda n.n(t + i + a + s) + t + i + s$$



Depends on size of input

Beyond intersection, T2 will always win

Asymptotic Notation

Consider two functions $f(n)$ and $g(n)$ with integer inputs and numerical outputs

We say f grows no faster then g in the limit if:

There exist positive constants c and n_0 such that

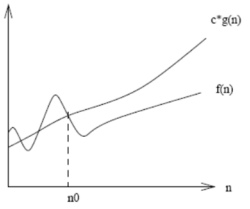
$$f(n) \le c \cdot g(n) \quad \forall n \ge n_0$$

We write this as: $f(n) = O(g(n))$

Read as "f is Big Oh of g"

We can also say "f is ***asymptotically dominated*** by g"

"g is an upper bound on f"
"f grows no faster than g"



49

Definition of Big Oh

Formal definition:

$$f(n) = O(g(n)) \text{ iff } \exists c \in R_+, n_0 \in N \bullet (\forall n \in N \bullet n \ge n_0 \Rightarrow f(n) \le c \cdot g(n))$$

Breaking this up:

$\exists \ldots n_0 \in \mathbb{N} \ldots n \ge n_0$ Means we don't care about small n

$\exists c \in \mathbb{R}^+ \ldots f(n) \le c \cdot g(n)$ Means we don't care about constant speedups

Unusual notation: "one way equality"

Really an ordering relation (think of < and >)

$f(n) = O(g(n))$ **definitely does not imply:** $O(g(n)) = f(n)$

50

Definition of Big Oh

Might like to think in terms of sets:

$$O(g(n)) = \{f(n) : \exists c \in \mathbb{R}_+, n_0 \in \mathbb{N} \bullet (\forall n \in \mathbb{N} \bullet n \ge n_0 \Rightarrow f(n) \le c \cdot g(n))\}$$

In this way, we can interpret $f(n) = O(g(n))$ as $f(n) \in O(g(n))$

Sometimes read as "f is in Big Oh of g"

51

Big Oh Example

$\lambda_n \cdot n^2 + 1 = O(\lambda_n \cdot n^2)$ - True or false?

How would we prove it?

Traditional to drop λ_n.

So we write: $n^2 + 1 = O(n^2)$

Go back to definition:

$f(n) = O(g(n))$ iff $\exists c \in \mathbb{R}^+, n_0 \in \mathbb{N} \bullet (\forall n \in \mathbb{N} \bullet n \ge n_0 \Rightarrow f(n) \le c \cdot g(n))$

To prove $\exists x \bullet P$ we need:

- a ***witness*** (value) for x
- a ***proof*** that P holds when witness substituted for x

52

Big Oh Example

$n^2 + 1 = O(n^2)$

Let's choose: $c = 2$

Need to find an n_0 such that:

$\forall n \in \mathbb{N} \bullet n \ge n_0 \Rightarrow n^2 + 1 \le 2n^2$

In this case, $n_0 = 1$ or greater will do

By convention, always complex to simple:

$$\text{complex} = O(\text{ simple })$$

e.g. $3n^2 + 102n - 56 = O(n^2)$
$3n^2 + 102n - 56 \neq O(n^3)$
$3n^2 + 102n - 56 \neq O(n)$

Related operators follow from definition of Big Oh…

53

Complexity Theory for Engineers

- Properties of Big Oh and others leads to mechanical rules for simplification
  - Drop low order terms
  - Ignore leading constants

$3n^3 + 90n^2 - 5n + 6046$

54

Complexity Theory for Engineers

- Properties of Big Oh and others leads to mechanical rules for simplification
  - Drop low order terms
  - Ignore leading constants

$3n^3 + 90n^2 - 5n + 6046$

Complexity Theory for Engineers

- Properties of Big Oh and others leads to mechanical rules for simplification
  - Drop low order terms
  - Ignore leading constants

$3n^3 + 90n^2 - 5n + 6046$

Complexity Theory for Engineers

- Properties of Big Oh and others leads to mechanical rules for simplification
  - Drop low order terms
  - Ignore leading constants

$3n^3 + 90n^2 - 5n + 6046 = O(n^3)$

Complexity

- Also known as *Computational complexity*
- It is a *characterization of the time or space requirements* for solving a problem by a particular algorithm
  - Time complexity
    - The time complexity of a program/ algorithm is the *amount of computer time* that it needs to run for completion
  - Space complexity
    - The space complexity of a program/ algorithm is the *amount of memory* that it needs to run for completion
- The complexity of an algorithm is the function which gives the *running time and/ or space in terms of the input size*.

Time Complexity

- While measuring the time complexity of an algorithm, we concentrate on developing only the *frequency count for all key statements* (statements that are important and are the basic instructions of an algorithm)

Examples

- Algorithm A

```
a = a+1
```

  - (frequency count of algo A is 1)
- Algorithm B

```
for x=1 to n
        a = a+1
loop
```

  - (frequency count of algo B is *n*)
- Algorithm C

```
for x=1 to n
        for y=1 to n
                a = a+1
loop
```

  - (frequency count of algo C is n^2)
- ***Note:*** Determine only those statements which may have the greatest frequency count

Linear Search

Linear array — Given element

LINEAR (ARR, N, ITEM, LOC)

(The algorithm finds the location LOC of ITEM in ARR, or sets LOC=0 if the search is unsuccessful)

1. [Insert ITEM at the end of ARR]
   Set ARR[N+1] = ITEM
2. [Initialize counter] Set LOC = 1
3. [Search for ITEM]
   Repeat while ARR[LOC] ≠ ITEM
   Set LOC = LOC+1
   [End of loop]
4. [Successful ?] If LOC = N+1, then Set LOC = 0
5. Exit

Complexity of Linear Search

It is measured by the number $f(n)$ of comparisons required to find ITEM in ARR where ARR contains n elements

- Worst Case:
  $f(n) = n+1$ (when ITEM does not appear in ARR)
  $= O(n)$
- Average Case:
  The number of comparisons can be any of the numbers 1, 2, 3, …n and each number occurs with probability $p = 1/n$, then
  $f(n) = 1\times1/n + 2\times1/n + \ldots + n\times1/n = \Sigma n \times 1/n = (n+1)/2$
  $\approx$ half the number of elements in the array
  $= O(n)$

Space Complexity

- The space needed by the program is the sum of the following components:
  - *Fixed space requirement:* This includes the instruction space, for simple variables, fixed size structured variables, and constants.
  - *Variable space requirement:* This consists of space needed by structured variables whose size depends on particular instance of variables. *It also includes the additional space required when the function uses recursion.*

Time-Space Tradeoff

- Sometimes the choice of data structure involves a time-space tradeoff:

  by increasing the amount of space for storing the data, one may be able to reduce the time needed for processing the data and vice versa

Time-Space Tradeoff

- In computer science, a **space-time** or **memory-time tradeoff** is a situation where the *memory* use can be reduced at the cost of slower program execution, or vice versa or the computation time can be reduced at the cost of increased memory use. As the relative costs of CPU cycles, RAM space, and hard drive space change — hard drive space has, for some time, been getting cheaper at a much faster rate than other components of computers, the appropriate choices for space-time tradeoffs have changed radically.
- By exploiting a space-time tradeoff, a program can be made to run much faster.

Time-Space Tradeoff

- The most common situation is an algorithm involving a *lookup table*: an implementation can include the entire table, which reduces computing time, but increases the amount of memory needed, or it can compute table entries as needed, increasing computing time, but reducing memory requirements.
- A space-time tradeoff can be applied to the problem of *data storage*. If data is stored uncompressed, it takes more space but less time than if the data were stored compressed (since compressing the data reduces the amount of space it takes, but it takes time to run the *compression algorithm*). Depending on the particular instance of the problem, either way is practical.